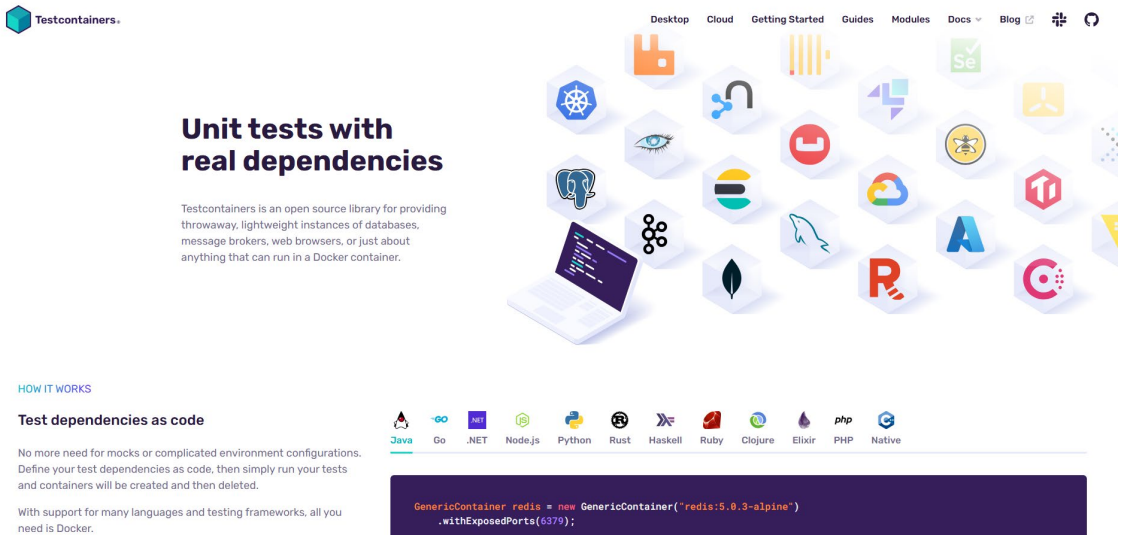
1 - Integration tests

Integration testing verifies that different parts of an application work together correctly, often involving real external dependencies like databases or message brokers. Unlike unit tests, integration tests check the interaction between components in a realistic environment.

Testcontainers is a Java library that helps create lightweight, throwaway instances of external services (like databases) using Docker containers during tests. This ensures tests run against real, isolated environments without manual setup, improving reliability and consistency.

Using integration tests with Testcontainers allows developers to automate end-to-end testing with real infrastructure components, leading to more robust and maintainable applications.

https://testcontainers.com/



Add dependencies to pom.xml to allow integration tests with test container.

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-testcontainers</artifactId>
        <scope>test</scope>
</dependency>
```

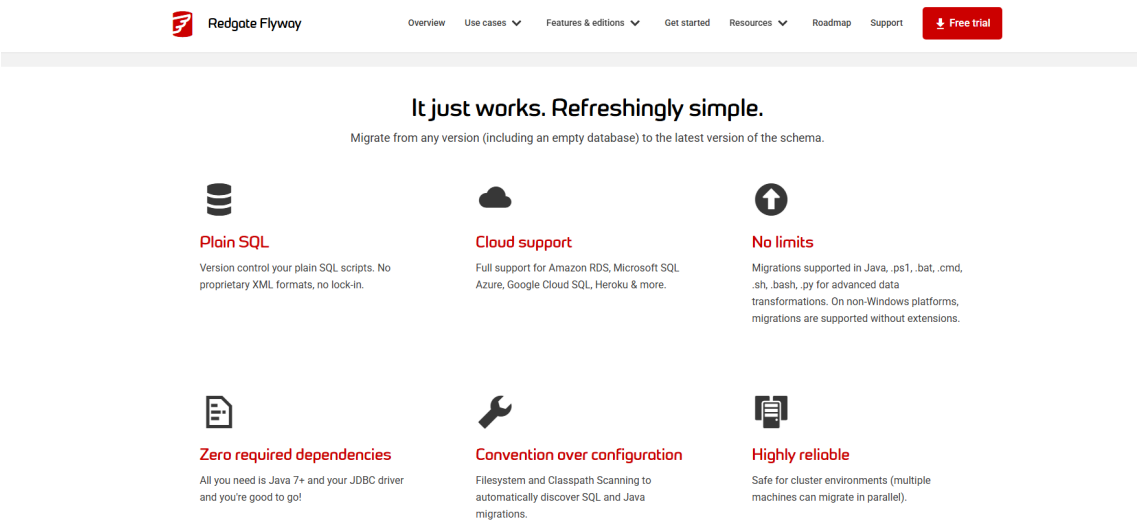
Add dependencies to allow rest communication during tests.

```
<dependency>
```

```
        <groupId>io.rest-assured</groupId>
        <artifactId>rest-assured</artifactId>
</dependency>
```

2 - Flyway

https://www.red-gate.com/products/flyway/community/



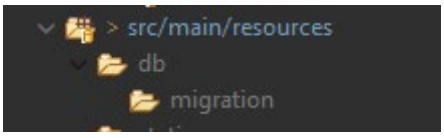
Flyway is a lightweight and powerful database migration tool. It helps manage and automate version control for your database schema. Flyway uses simple SQL or Java-based migration scripts to apply incremental changes to your database in a controlled and consistent way.

It integrates easily with Spring Boot and supports many relational databases like MySQL, PostgreSQL, Oracle, and more. With Flyway, you can ensure your database structure stays in sync across environments (development, testing, production) and team members.

Add the dependencies to enable flyway db migration

```
<dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-mysql</artifactId>
</dependency>
```

Whenever flyway is activated, it searches for the migration scripts in the next folder.



The scripts for migrations must follow the next name convention:

V<number>__<name>.sql → Examples: V1__orders.sql, V1__init.sql

3 - Spring Cloud Open Feign

Spring Cloud OpenFeign is a declarative HTTP client that makes it easy to call other microservices in a Spring Boot application.

◆ **What it does:**

It allows you to define Java interfaces to call REST APIs, avoiding boilerplate code with RestTemplate or WebClient.

◆ **How it works:**

Add the dependency:

spring-cloud-starter-openfeign

Enable Feign clients in your main class with @EnableFeignClients.

Create an interface with @FeignClient, specify the service name or URL, and define the endpoints using Spring annotations like @GetMapping.

Inject and use the interface as a regular Spring bean.

✅ **Benefits:**

- Simple and readable code
- Built-in integration with Spring Boot
- Supports load balancing, fallback, logging, and more

4 - WireMock

WireMock is a tool used to mock HTTP services, allowing you to simulate external APIs and test your application without depending on real remote services.

1. Add the WireMock dependency to your project (order-related module) (spring-cloud-starter-contract-stub-runner)
2. Use the @AutoConfigureWireMock(port = ...) annotation to configure WireMock on a specified port for integration testing

```
<!--
    Enables the use of stubbed responses from external services during tests.
    Useful when your app depends on other microservices — you can simulate their responses
    without having to run the actual service.
    Internally uses WireMock to spin up a fake HTTP server during tests.
-->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-contract-stub-runner</artifactId>
    <scope>test</scope>
</dependency>
<!--
    Library for testing REST APIs in a fluent and readable way.
    Allows you to easily send HTTP requests (GET, POST, etc.) and assert status, headers, and body.
-->
<dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
</dependency>
<!-- Open feign dependency to allow communication between microservices -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

5 - API Gateway

An API Gateway is a server that acts as a single entry point for all client requests to a system made up of multiple services (usually microservices). It receives the requests, forwards them to the correct service, and returns the response back to the client.

Dependency:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway-server-webmvc</artifactId>
</dependency>
```

✅ Benefits:

Centralized entry point – Simplifies how clients access the system.

Security and monitoring – Easier to implement authentication, rate limiting, and logging.

Request routing – Routes traffic to the right service based on URL, method, etc.

Reduces client complexity – Clients don't need to know about service locations.

Request aggregation – Combines multiple service calls into one response.

❌ Drawbacks:

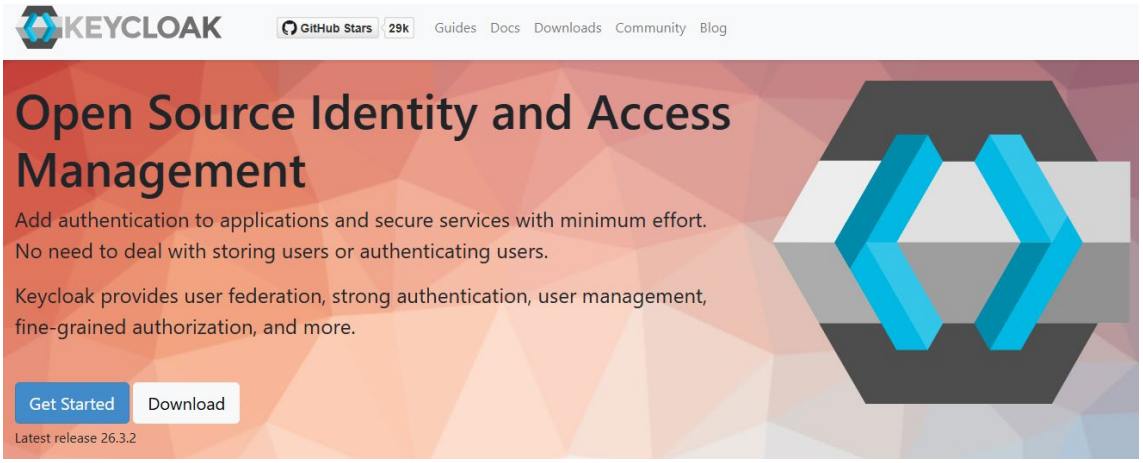
Single point of failure – If the gateway goes down, the whole system may become unreachable.

Increased latency – Adds an extra network hop between the client and services.

Complex configuration – Routing rules and policies can grow complex over time.

Scalability needed – The gateway must handle all traffic, so it must be well-scaled.

6 - KeyCloak

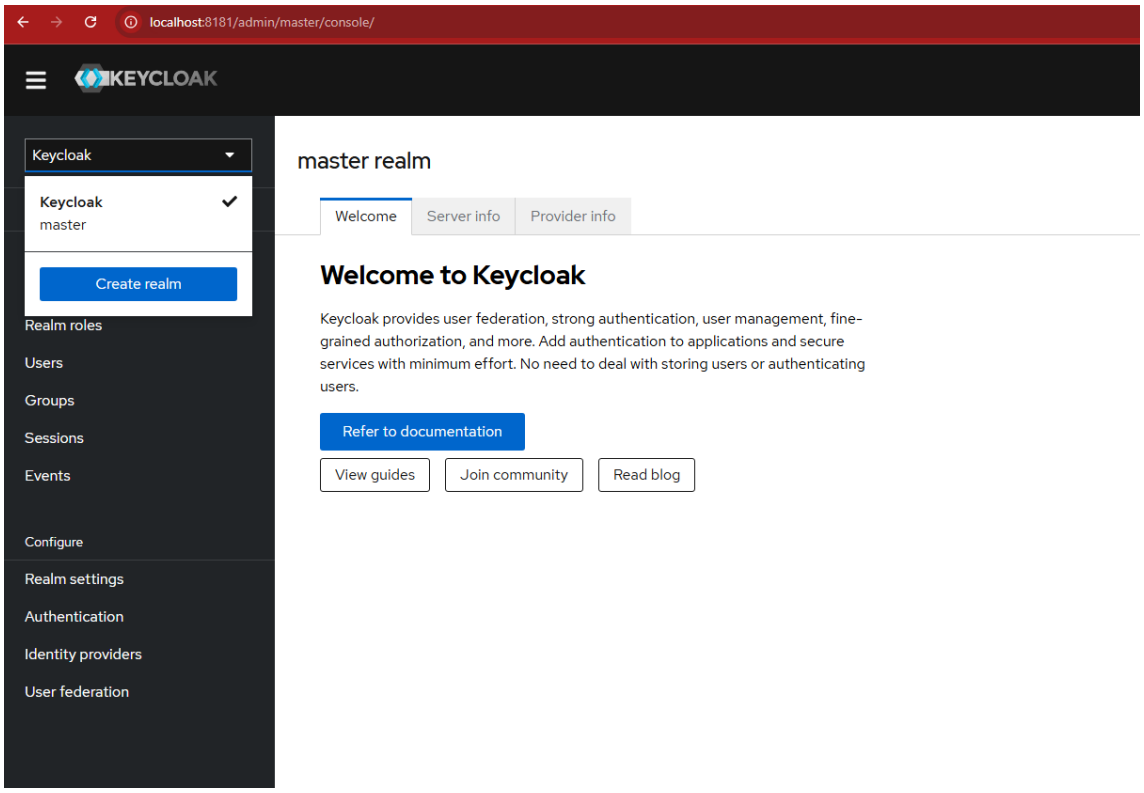


Keycloak is an open-source Identity and Access Management (IAM) solution developed by Red Hat. It provides features such as Single Sign-On (SSO), user federation, identity brokering, and centralized authentication and authorization for applications and services. With support for standard protocols like OAuth 2.0, OpenID Connect, and SAML 2.0, Keycloak enables secure and flexible user management out of the box. It also includes an easy-to-use admin console and integrates well with modern applications and microservice architectures.

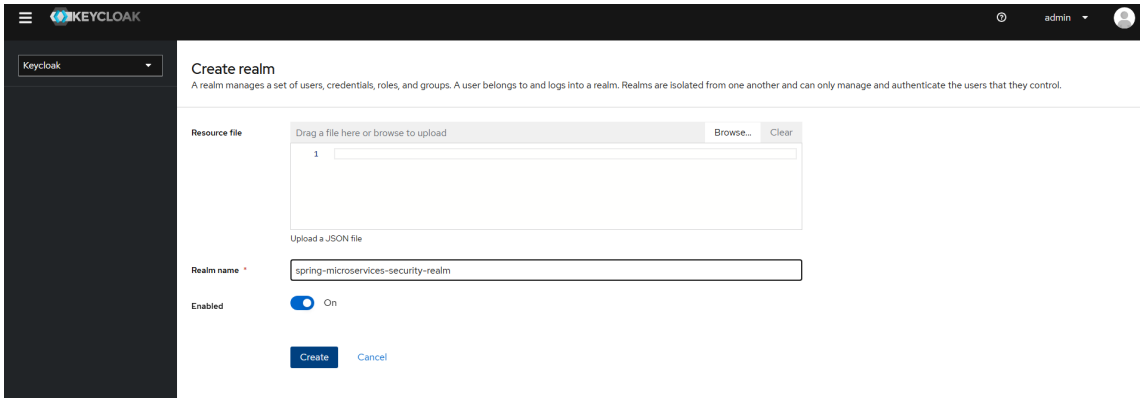
OAuth 2.0: It is an authorization protocol that allows applications to access a user's resources on another service (such as Google or Facebook) without needing to share the user's password.

OpenID Connect (OIDC): It is an identity layer built on top of OAuth 2.0. It allows applications to verify the user's identity and securely obtain basic profile information.

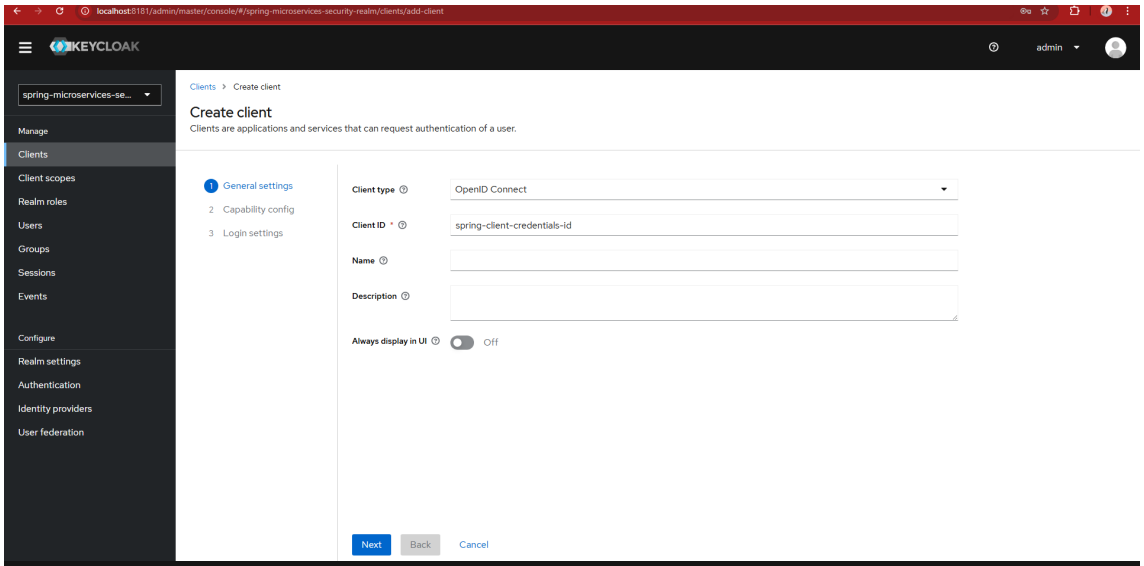
SAML (Security Assertion Markup Language): It is an XML-based authentication protocol used for Single Sign-On (SSO), especially in enterprise environments. It allows users to authenticate once and access multiple applications.

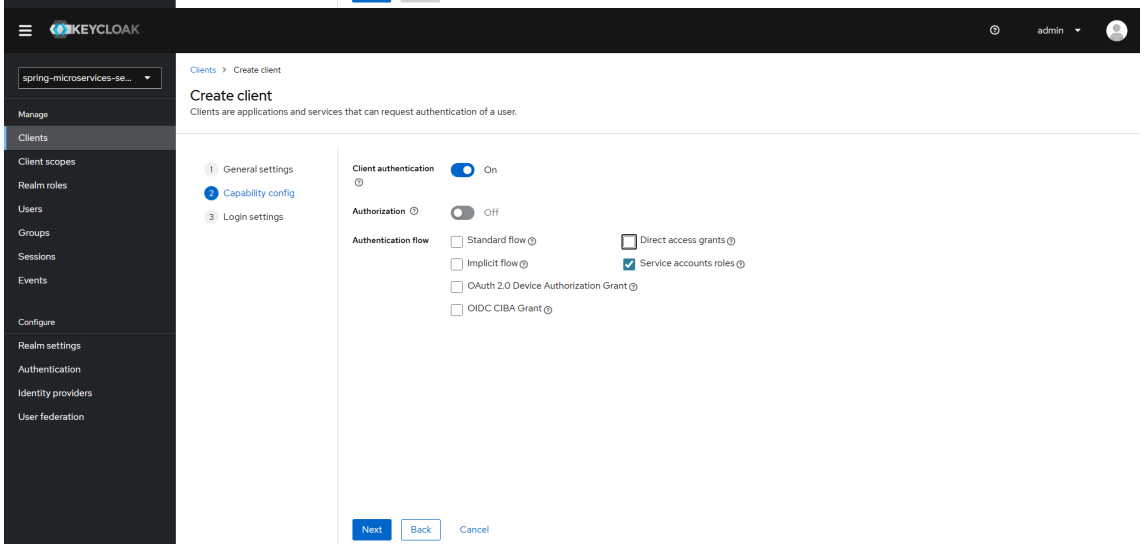


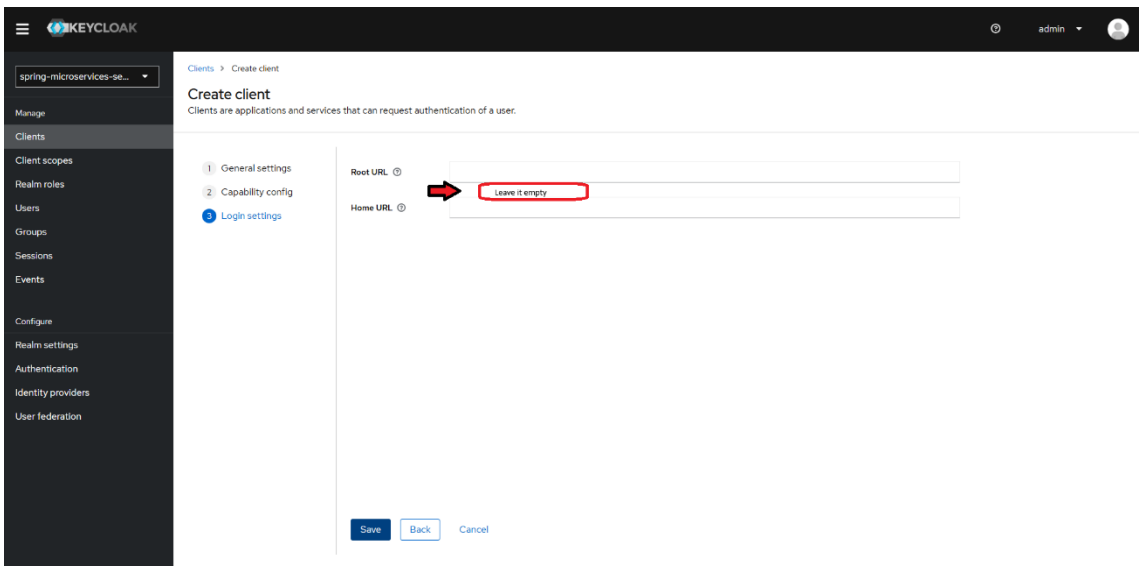
The first step is to create a new realm for the project configuration (users, roles, credentials etc).



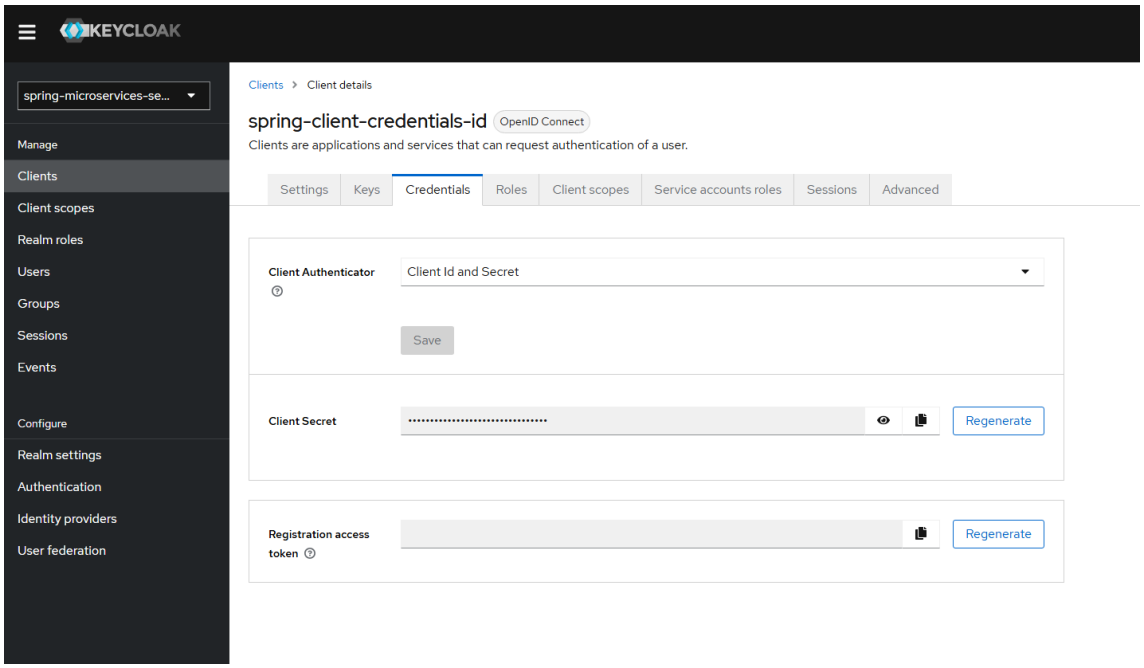
The next step is to create a new client


localhost:8181/admin/master/console/#/spring-microservices-security-realm/clients/add-client
KEYCLOAK
admin
spring-microservices-se...
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation
Clients > Create client
Create client
Clients are applications and services that can request authentication of a user.
1 General settings
2 Capability config
3 Login settings
Client type
OpenID Connect
Client ID *
spring-client-credentials-id
Name
Description
Always display in UI
Off
Next
Back
Cancel

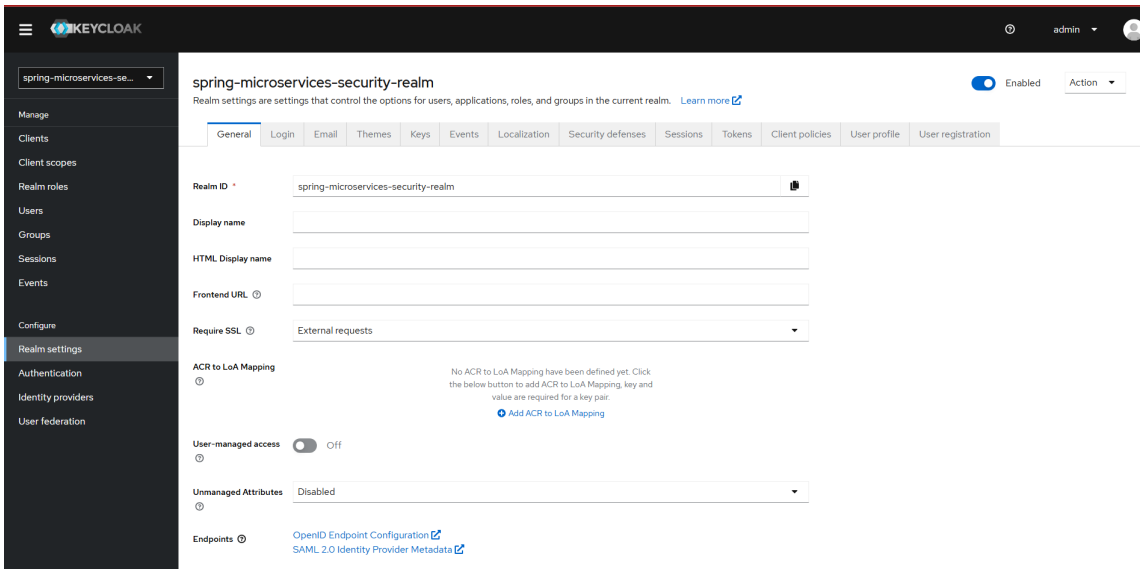

KEYCLOAK
admin
spring-microservices-se...
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation
Clients > Create client
Create client
Clients are applications and services that can request authentication of a user.
1 General settings
2 Capability config
3 Login settings
Client authentication
On
Authorization
Off
Authentication flow
Standard flow
Direct access grants
Implicit flow
Service accounts roles
OAuth 2.0 Device Authorization Grant
OIDC CIBA Grant
Next
Back
Cancel

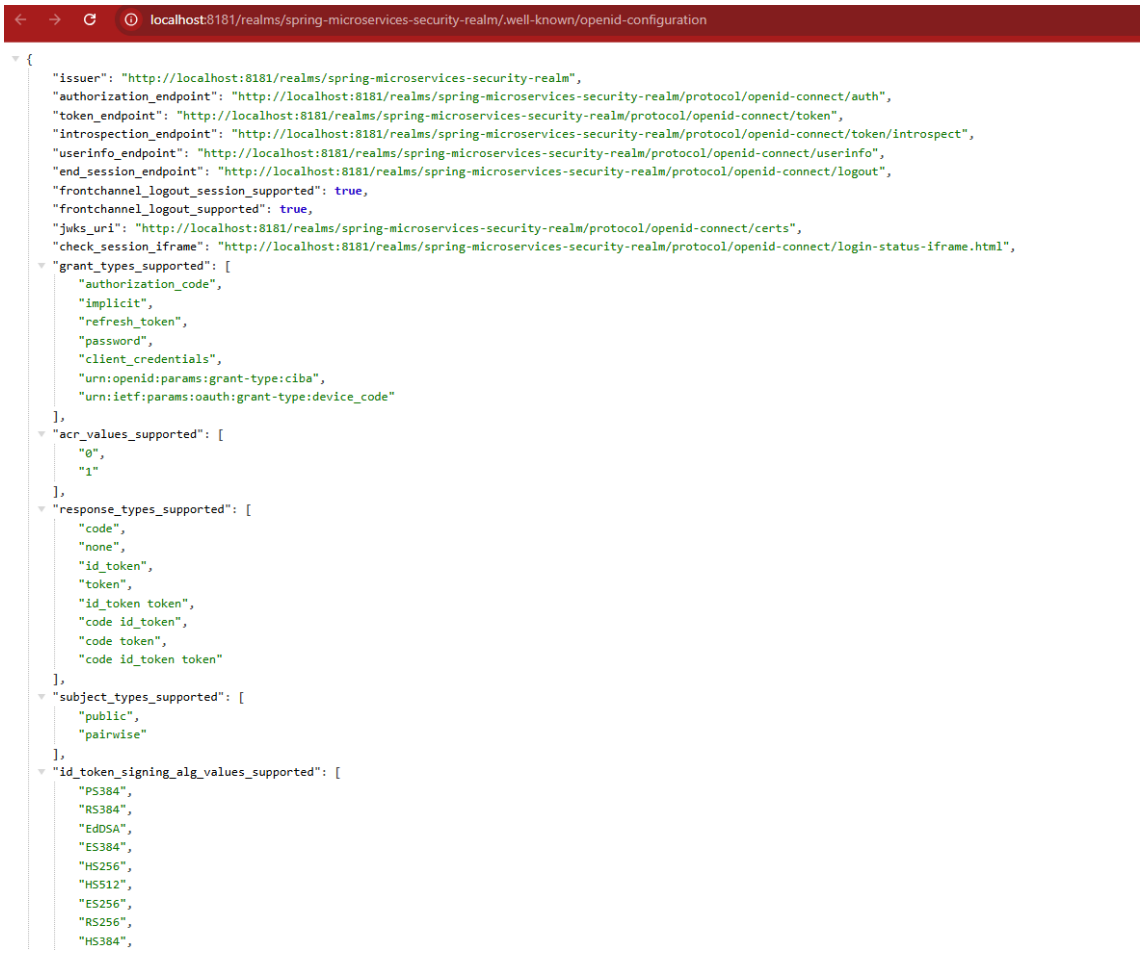

KEYCLOAK
admin
spring-microservices-se...
Manage
Clients
Client scopes
Realm roles
Users
Groups
Sessions
Events
Configure
Realm settings
Authentication
Identity providers
User federation
Clients > Create client
Create client
Clients are applications and services that can request authentication of a user.
1 General settings
2 Capability config
3 Login settings
Root URL
Home URL
Leave it empty
Save
Back
Cancel

Next, check the client secret



Through realm settings, we can access openId configuration endpoint

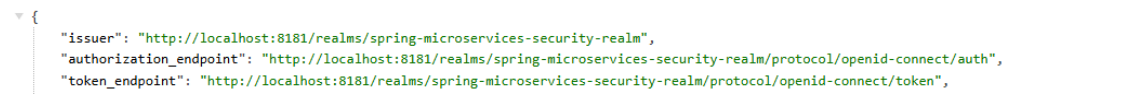




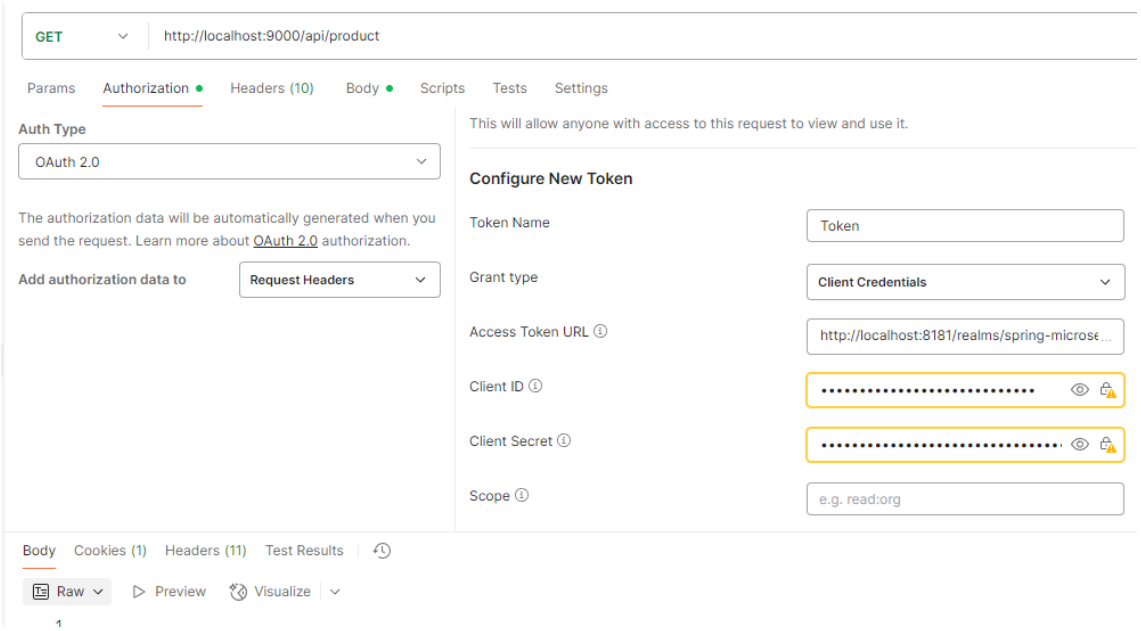
The API Gateway module will be configured as a resource server for oauth2.
Add the next dependency:

```
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-oauth2-resource-server</artifactId>
</dependency>
```

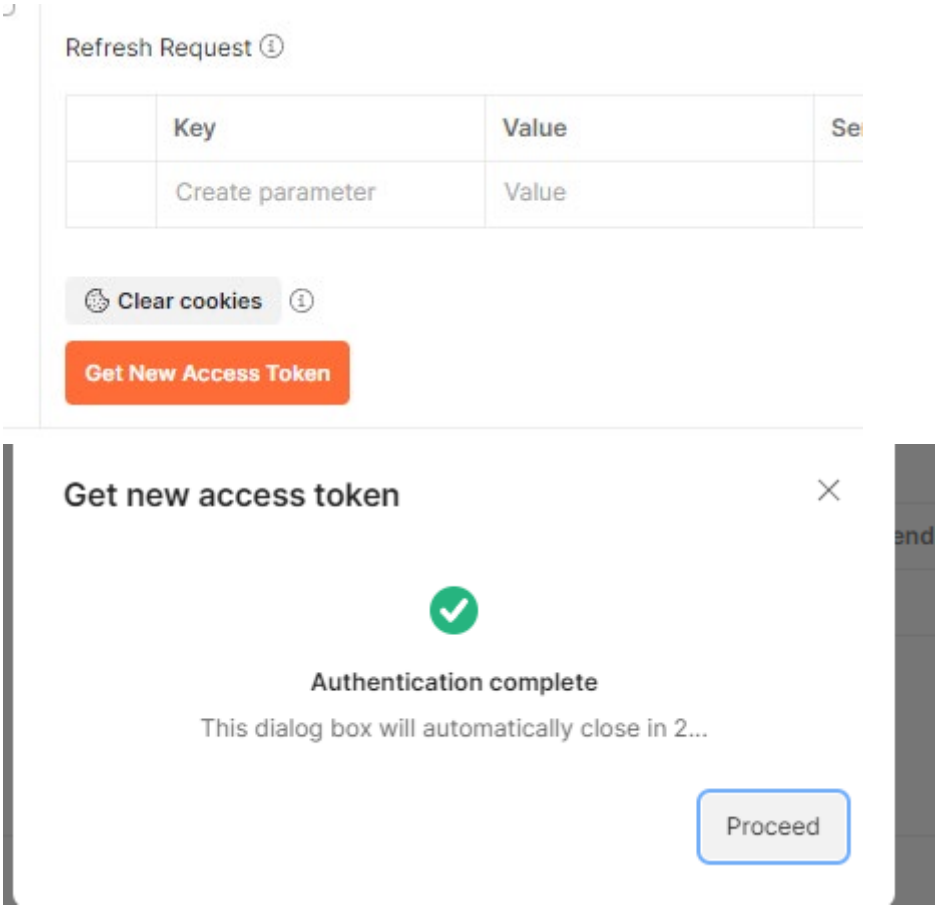
In the openId endpoint (keycloak / realm settings), we can check the token url

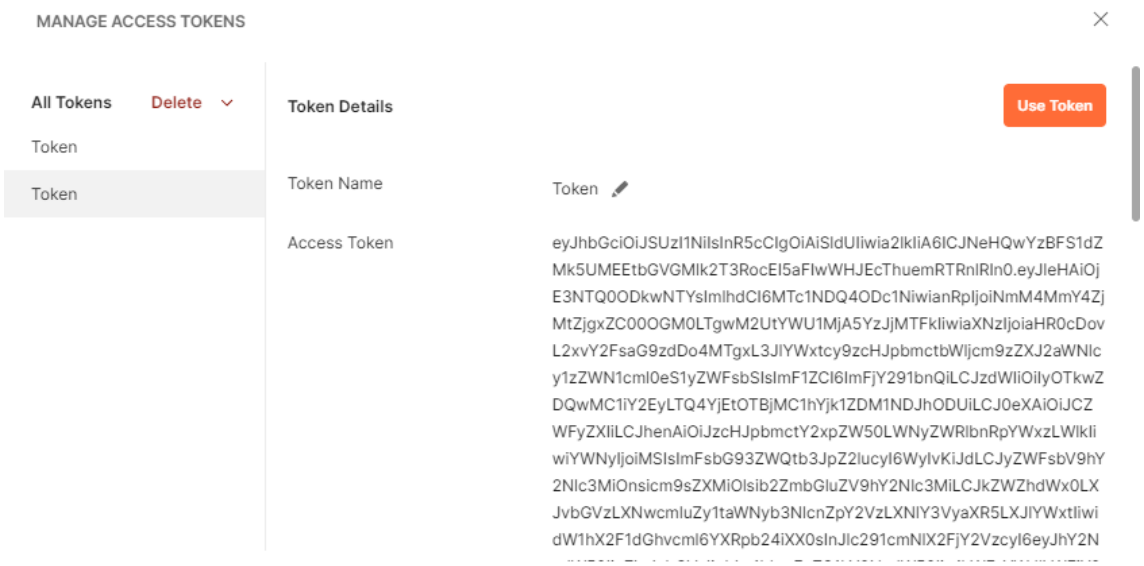


We can configure a new token for our requests in postman, so that we get authorized to access the endpoints of our application.
Besides the token url, we also need the client's ID and secret.

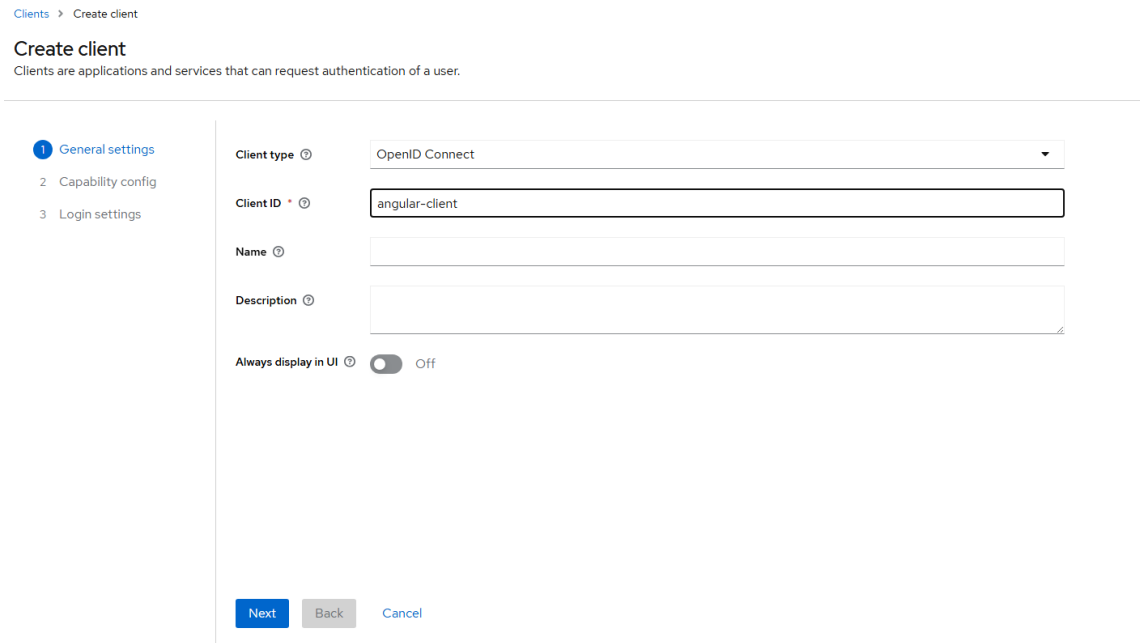


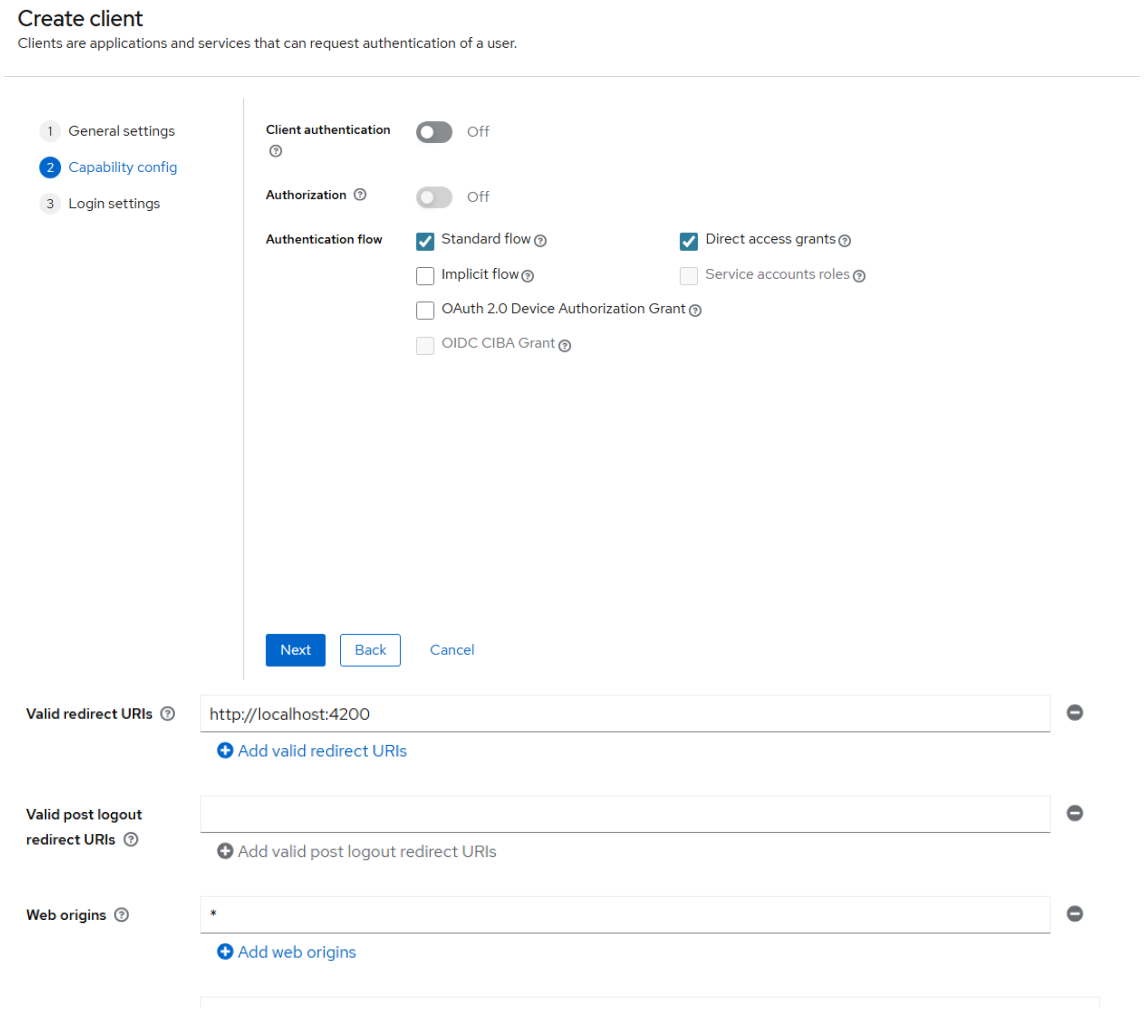
Finally scroll to the bottom and clic the button to generate the token, then clic on proceed.



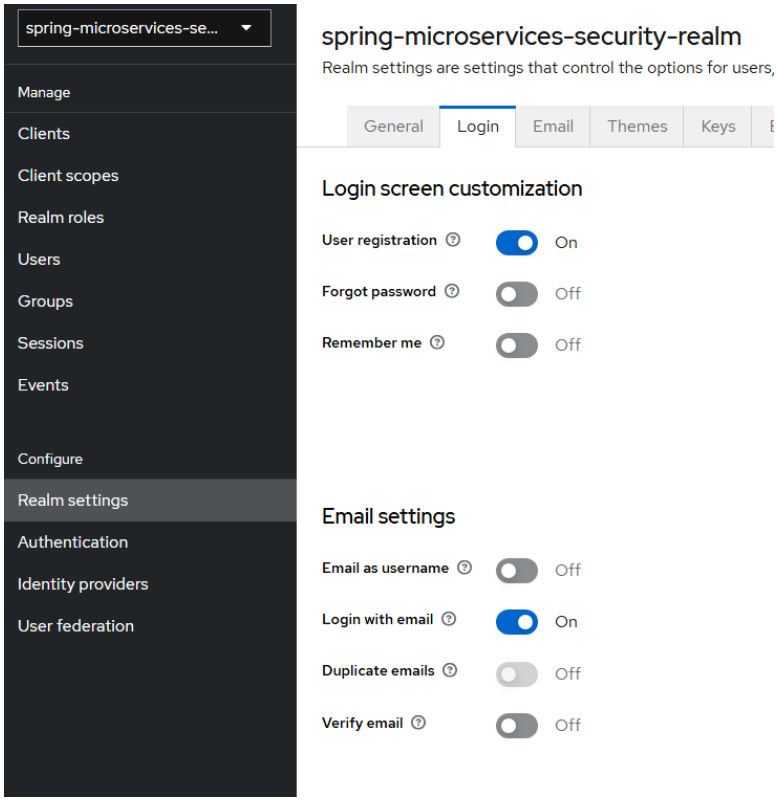

MANAGE ACCESS TOKENS
All Tokens
Delete
Token
Token
Token Details
Use Token
Token Name
Token
Access Token
eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUIiwia2lkIiA6ICJNeHQwYzBFS1dZ
Mk5UMEEtbGVGMlk2T3RocEI5aFIwWHJEcThuemRTRnlRln0.eyJleHAiOj
E3NTQ0ODkwNTYsImlhdCI6MTc1NDQ4ODc1NiwianRpIjoiNmM4MmY4Zj
MtZjgxZC00OGM0LTgwM2UtYWU1MjA5YzJjMTFkIiwiaXNzIjoiaHR0cDov
L2xvY2FsaG9zdDo4MTgxL3JlYWxtcy9zcHJpbmctbWljcm9zZXJ2aWNlc
y1zZWN1cml0eS1yZWFsbSIsImF1ZCI6ImFjY291bnQiLCJzdWIiOiIyOTkwZ
DQwMC1iY2EyLTQ4YjEtOTBjMC1hYjk1ZDM1NDJhODUiLCJ0eXAiOiJCZ
WFyZXIiLCJhenAiOiJzcHJpbmctY2xpZW50LWNyZWRlbnRpYWxzLWlkli
wiYWNyIjoiMSIsImFsbG93ZWQtb3JpZ2lucyI6WyIvKiJdLCJyZWFsbV9hY
2Nlc3MiOnsicm9sZXMiOlsib2ZmbGluZV9hY2Nlc3MiLCJkZWZhdWx0LX
JvbGVzLXNwcmluZy1taWNyb3NlcnZpY2VzLXNlY3VyaXR5LXJlYWxtliwi
dW1hX2F1dGhvcml6YXRpb24iXX0sInJlc291cmNlX2FjY2VzcyI6eyJhY2N
Press on use token and the token gets added to the header.

To authenticate the Angular frontend client application, we need to create a new client in our Keycloak realm.

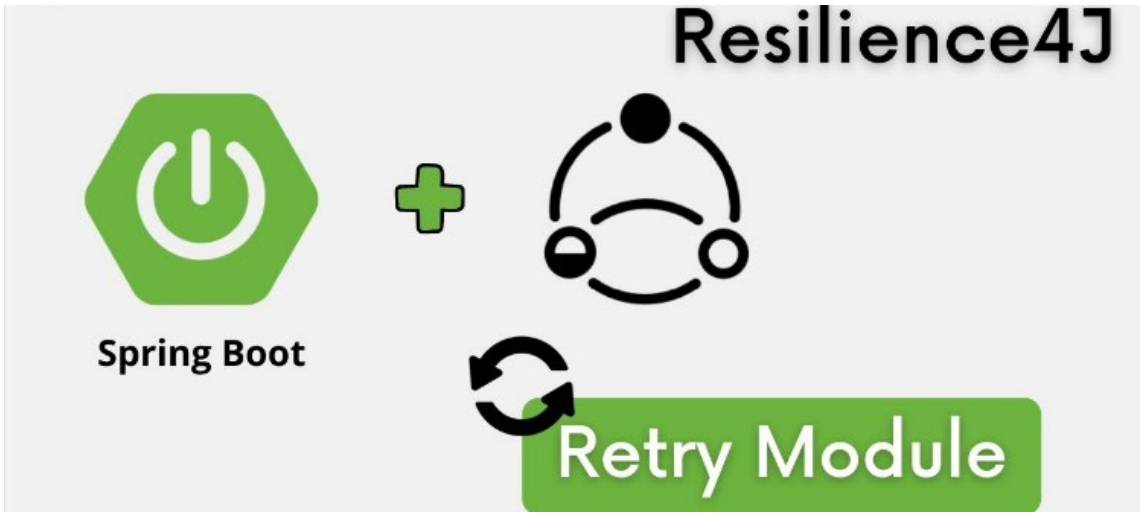




We can also activate the user registration option in our realm settings.



7 - Circuit breaker



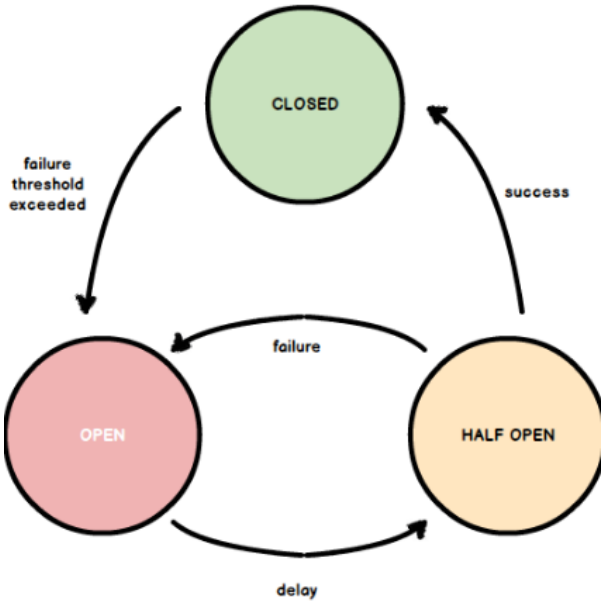
A **circuit breaker** is a design pattern used in microservices and distributed systems to improve reliability. It acts like a safety switch that stops requests to a failing service after a certain number of errors, preventing system overload and long wait times. After a cooldown period, it tests the service again to see if it has recovered, allowing requests to resume. This helps maintain overall system stability and resilience.

Resilience4j is a lightweight, easy-to-use fault tolerance library for Java applications. It

helps developers build resilient systems by providing common patterns like **circuit breakers**, **rate limiters**, **retry mechanisms**, and **bulkheads**. Resilience4j is designed to work well with modern Java frameworks like Spring Boot, allowing you to protect your microservices from failures and improve overall system stability.

We will apply the circuit breaker pattern across all microservices.

These are the 3 posible states for the circuit breaker



- **Closed** ✓

All requests are allowed. If the failure rate exceeds the threshold (e.g., 50%) based on the last N calls, the state transitions to **Open**.

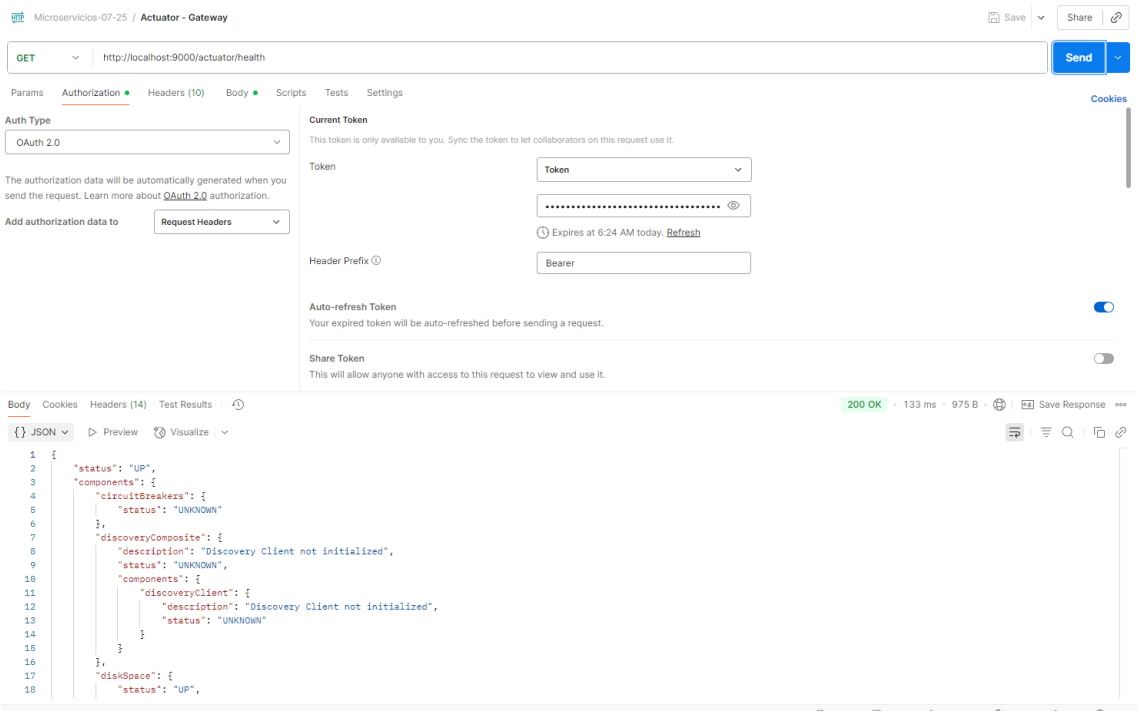
- **Open** ✗

No requests are allowed. The Circuit Breaker waits for a configured **cooldown period** (`waitDurationInOpenState`). After that, it moves to **Half-Open**.

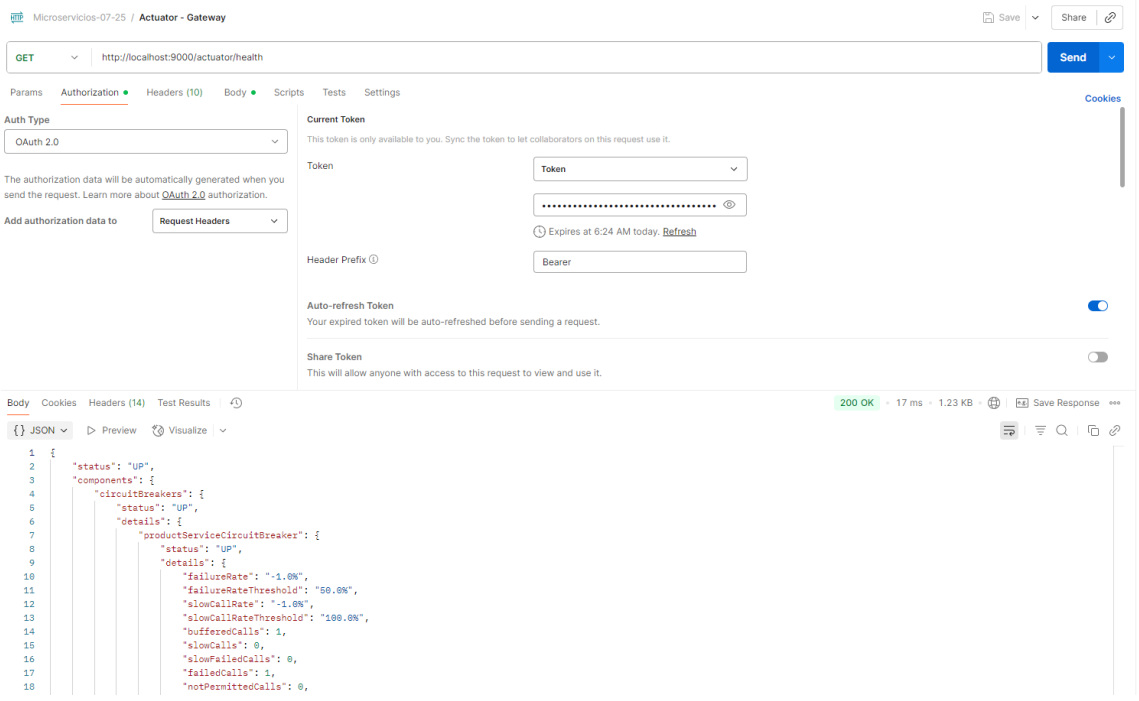
- **Half-Open** ⚠

A limited number of test requests are allowed:

  - If they succeed → the breaker **closes** (healthy again).
  - If any fail → it goes back to **Open**.



If we stop one of our services, and try to send a request, the circuit breaker will update its status to “UP”, and we will be able to see it through the actuator endpoint



After enabling the actuator and the circuit breaker, we can monitor various metrics through the actuator endpoint, including the circuit breaker status.
Since we configured the circuit breaker to require a minimum of 5 failed requests before changing its status to OPEN, if we force 5 requests while the service is down and then access the actuator endpoint, we can verify that the status is now "OPEN".

```
        "circuitBreakers": {
            "status": "UNKNOWN",
            "details": {
                "productServiceCircuitBreaker": {
                    "status": "CIRCUIT_OPEN",
                    "details": {
                        "failureRate": "100.0%",
                        "failureRateThreshold": "50.0%",
                        "slowCallRate": "0.0%",
                        "slowCallRateThreshold": "100.0%",
                        "bufferedCalls": 5,
                        "slowCalls": 0,
                        "slowFailedCalls": 0,
                        "failedCalls": 5,
                        "notPermittedCalls": 0,
                        "state": "OPEN"
                    }
                }
```

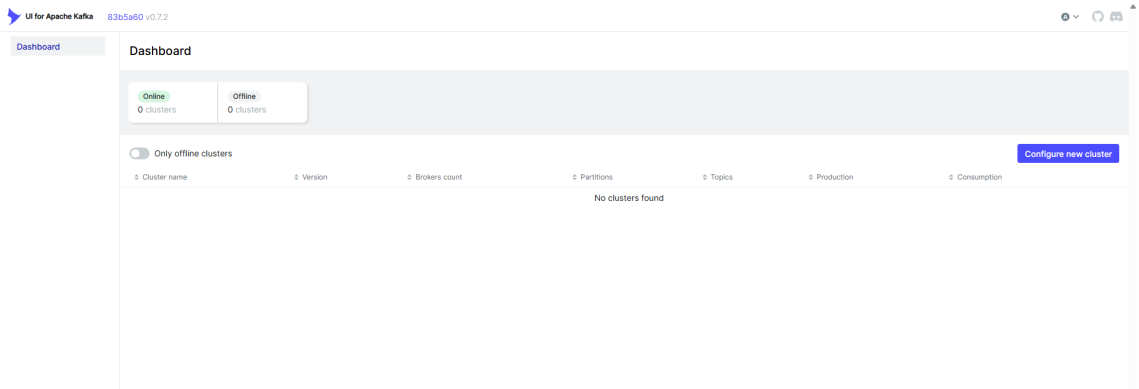

After the configured delay has passed, the status will be set to **HALF_OPEN**.
If **5 valid requests** are sent during this period, the status will be updated to **CLOSED** (it will be updated on the **fifth** valid request).

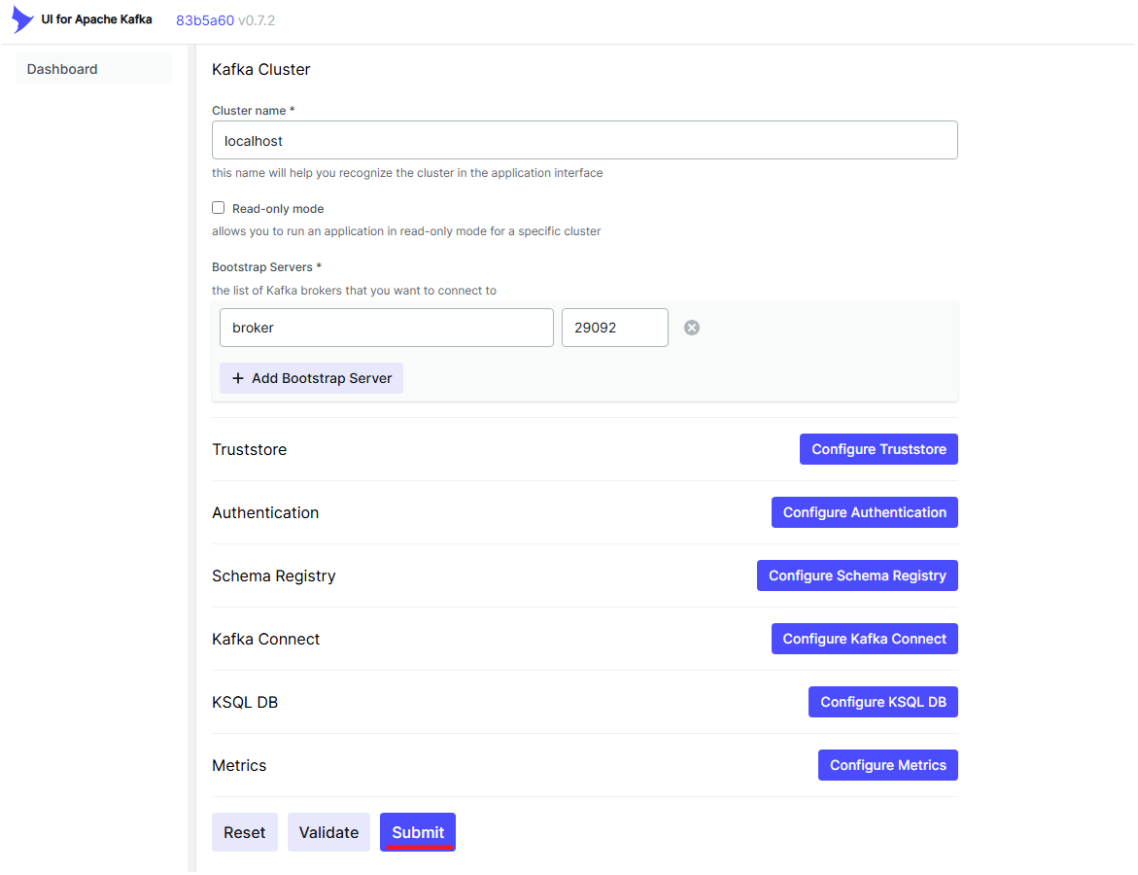

```
    "circuitBreakers": {
        "status": "UNKNOWN",
        "details": {
            "productServiceCircuitBreaker": {
                "status": "CIRCUIT_HALF_OPEN",
                "details": {
                    "failureRate": "-1.0%",
                    "failureRateThreshold": "50.0%",
                    "slowCallRate": "-1.0%",
                    "slowCallRateThreshold": "100.0%",
                    "bufferedCalls": 0,
                    "slowCalls": 0,
                    "slowFailedCalls": 0,
                    "failedCalls": 0,
                    "notPermittedCalls": 0,
                    "state": "HALF_OPEN"
                }
            }
```

8 - KAFKA

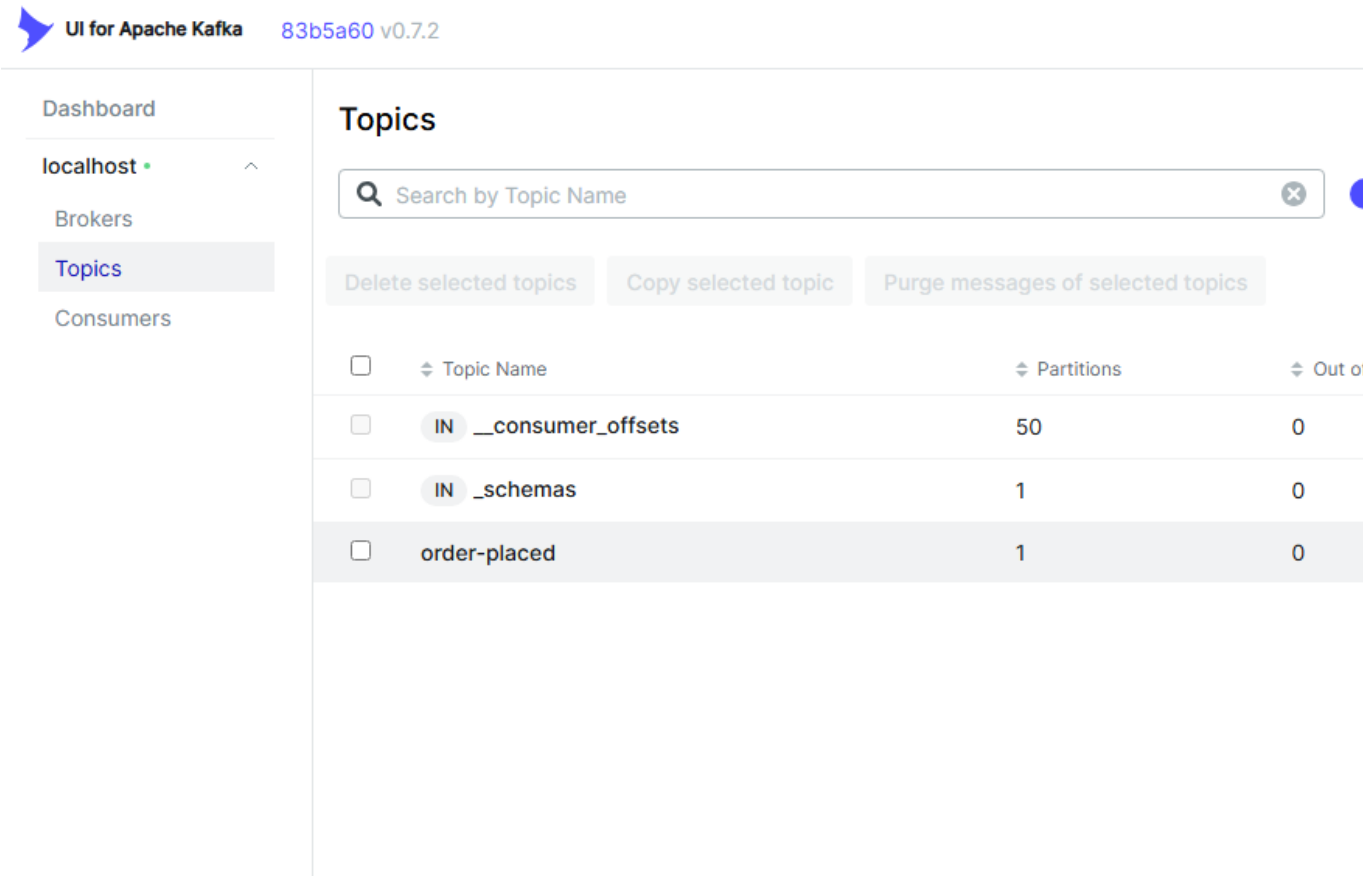
Apache Kafka is an open-source, distributed event streaming platform used for building real-time data pipelines and applications.
It works like a high-performance messaging system where producers send messages (events) to topics, and consumers read those messages.
Kafka is designed for high throughput, scalability, and fault tolerance, making it ideal for microservices communication, log aggregation, and real-time analytics.

Go to localhost:8086 (dockerfile) to access to kafka ui dashboard and configure a new cluster.





The cluster now appears in the dashboard, and we can access to “Topics”.
If we already made an order, we will see the order-placed topic created.



We can click on the Messages tab and check the message and its preview

Topics / order-placed

Overview | Messages | Consumers | Settings | Statistics

Seek Type: Offset | Offset

Partitions: All items are selected.

Key Serde: String

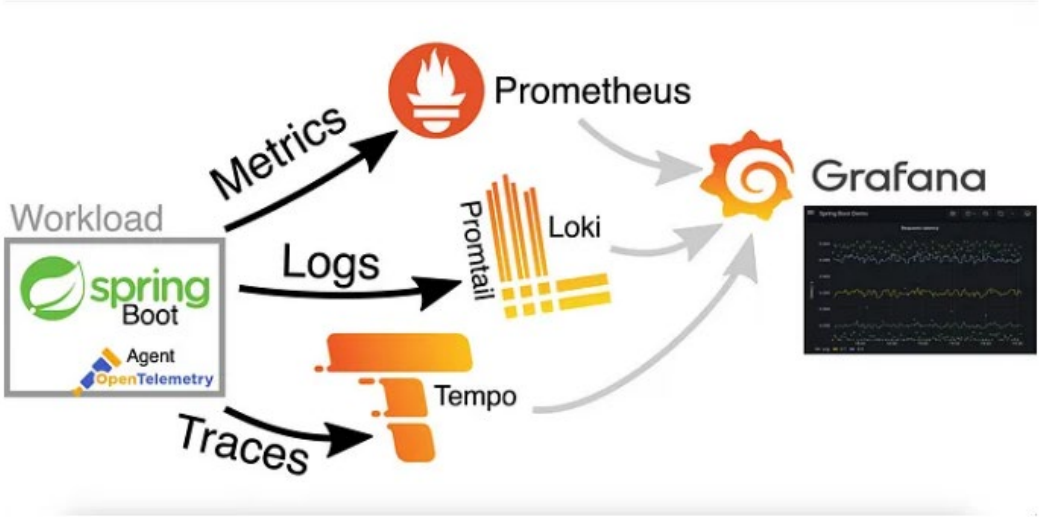
Search

+ Add Filters

	Offset	Partition	Timestamp	Key Preview
+	0	0	13/8/2025, 6:16:16	

← Back Next →

9 - Grafana stack



The **Grafana stack** is a set of observability tools for monitoring, logging, and tracing in modern applications. It includes:

- **Prometheus**: A metrics collection and monitoring system that scrapes numeric time-series data from applications and systems.
- **Grafana**: A visualization and dashboard platform that lets you query, visualize, and alert on data from Prometheus and other sources.
- **Loki**: A log aggregation system optimized for storing and querying logs alongside metrics without indexing full log content.
- **Tempo**: A distributed tracing backend that collects traces from applications to help debug and understand request flows.

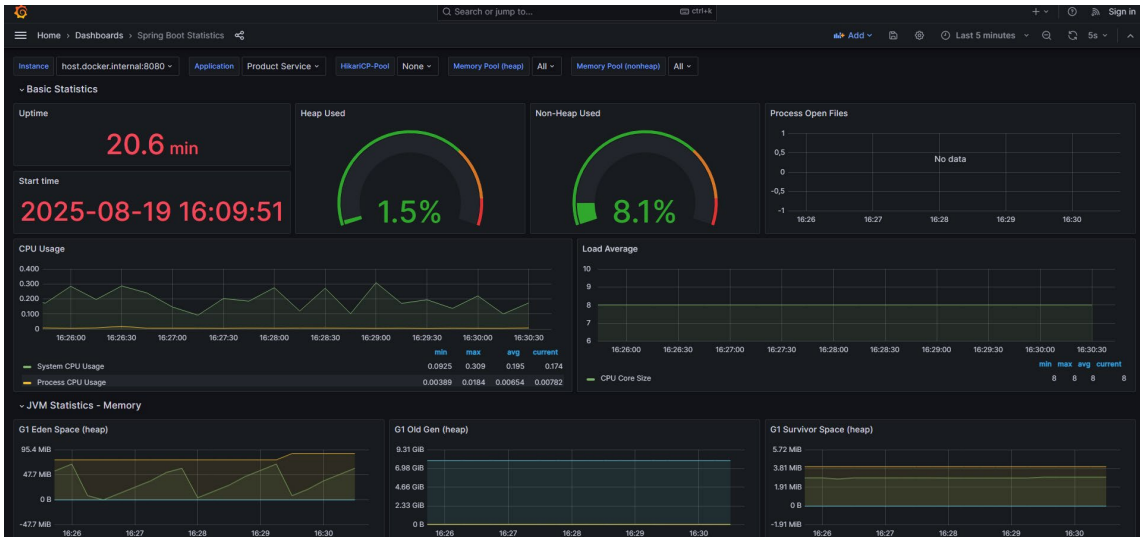
Together, Grafana, Loki, and Tempo provide **metrics, logging, and tracing** in a single ecosystem, making it easier to gain full observability over modern cloud-native systems.

- **Prometheus** → It collects metrics from your microservices (for example, how many requests each endpoint receives, how long they take, failures, etc.).

- **Grafana** → It's the visual interface where you see those metrics, logs, and traces; basically, it's the tool for monitoring and dashboards.

- **Loki** → It's like a "Prometheus for logs": it stores all the logs from your services and organizes them with labels so you can search and filter them from Grafana.

- **Tempo** → It's for tracing: it stores the traces of requests traveling through your microservices so you can see the full flow of a request and how long each microservice takes.



10 - Kubernetes

Kubernetes (K8s) is an **open-source platform** that automates the deployment, scaling, and management of **containerized applications**.

Why use it?

- Automatically runs and manages containers across many machines.
- Scales apps up or down based on demand.
- Replaces or restarts containers if they fail.
- Provides load balancing and service discovery.

Key concepts

- **Pod**: One or more containers running together.
- **Deployment**: Manages pods and updates.
- **Service**: Gives stable networking to pods.

How it works

1. Build your app into a **Docker image**.
2. Push the image to a **registry**.
3. Deploy it with **Kubernetes manifests**.
4. Kubernetes runs and manages your containers automatically.

Cloud Native Buildpacks

Cloud Native Buildpacks automatically **convert your source code into container images** without writing Dockerfiles.

Key points

- Detects your app's language and dependencies.
- Builds a **ready-to-run container image**.
- Works well with platforms like **Kubernetes** and **Cloud Foundry**.
- Helps **standardize and simplify** container builds for developers.

We can build Docker images using Maven (via the Spring Boot plugin) with the following command:

```
mvn clean spring-boot:build-image -DskipTests
```

kind (Kubernetes IN Docker) is a tool for running local Kubernetes clusters using Docker containers. It is mainly designed for testing and development purposes, allowing you to create lightweight, disposable clusters quickly